

Shiv Nadar University Chennai
CS3807 – Deep Learning Laboratory
Experiment 3 - Amrut Anand - Roll: 24011101003
Implementation of Convolutional Neural Networks (CNNs) for Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026– 27

1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

2. Background Theory

A Convolutional Neural Network consists of

- Convolution Layer
- Activation Layer
- Pooling Layer
- Fully Connected Layer

The convolution operation is

$$Y(i, j) = \sum_m \sum_n X(i + m, j + n) K(m, n)$$

where

- X : Input Image
- K : Kernel (Filter)
- Y : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

N Input Size

F Filter Size

P Padding

S Stride

2.1 Common Convolution Kernels

A **kernel** (or **filter**) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

1. Identity Kernel

The identity kernel preserves the original image without modifying its pixel values.

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Purpose:

- Preserves the original image.
- Used for understanding the convolution operation.

2. Sobel-X Kernel (Vertical Edge Detection)

The Sobel-X kernel detects **vertical edges** by measuring intensity changes along the horizontal direction.

$$\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$$

Applications:

- Object detection
- Lane detection
- Face recognition

3. Sobel-Y Kernel (Horizontal Edge Detection)

The Sobel-Y kernel detects **horizontal edges** by measuring intensity changes along the vertical direction.

$$\begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Text detection
- Boundary extraction
- Medical image analysis

4. Laplacian Kernel

The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Edge enhancement
- Satellite image analysis
- Medical imaging

5. Sharpening Kernel

This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

Applications:

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

6. Box Blur Kernel

The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Applications:

- Noise removal
- Image smoothing
- Image preprocessing

7. Gaussian Blur Kernel

Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Applications:

- Noise reduction
- Computer vision preprocessing
- Feature extraction

8. Emboss Kernel

The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{bmatrix} -2 & -1 & 0 \\ -1 & 1 & 1 \\ 0 & 1 & 2 \end{bmatrix}$$

Applications:

- Texture analysis
- Artistic image effects

9. Outline Detection Kernel

This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Applications:

- Image segmentation
- Boundary detection
- Shape extraction

10. Motion Blur Kernel

This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5} \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Applications:

- Motion simulation
- Image restoration
- Video processing

Summary of Common Kernels

Kernel	Size	Purpose	Applications
Identity	3×3	Preserve original image	Baseline comparison
Sobel-X	3×3	Detect vertical edges	Object detection
Sobel-Y	3×3	Detect horizontal edges	Boundary detection
Laplacian	3×3	Detect edges in all directions	Medical imaging
Sharpen	3×3	Enhance details	Image enhancement
Box Blur	3×3	Smooth image	Noise removal
Gaussian Blur	3×3	Reduce noise	Computer vision
Emboss	3×3	3D effect	Artistic filtering
Outline	3×3	Boundary extraction	Segmentation
Motion Blur	5×5	Simulate motion	Video processing

3. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Kernel

$$K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{bmatrix} 6 & 8 \\ 12 & 14 \end{bmatrix}$$

4. Numerical Example 2: Max Pooling

Feature map

$$\begin{bmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{bmatrix}$$

Using a 2×2 max pooling filter,
Window 1

$$\begin{bmatrix} 1 & 5 \\ 7 & 8 \end{bmatrix} \rightarrow 8$$

Window 2

$$\begin{bmatrix} 2 & 3 \\ 1 & 0 \end{bmatrix} \rightarrow 3$$

Window 3

$$\begin{bmatrix} 4 & 6 \\ 2 & 3 \end{bmatrix} \rightarrow 6$$

Window 4

$$\begin{bmatrix} 9 & 5 \\ 1 & 8 \end{bmatrix} \rightarrow 9$$

Output

$$\begin{bmatrix} 8 & 3 \\ 6 & 9 \end{bmatrix}$$

5. Numerical Example 3: Parameter Calculation

Suppose

Input Image

$$32 \times 32 \times 3$$

Convolution Layer

- Filters = 16
- Kernel = 3×3

Parameters

$$(3 \times 3 \times 3 + 1) \times 16$$

$$= (27 + 1) \times 16$$

$$= 448$$

Therefore,

Total trainable parameters = 448.

6. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

7. Experimental Procedure

Task 1

Load CIFAR-10.

- Display ten sample images.
- Print dataset dimensions.
- Plot class distribution.

Task 2

Implement a convolution layer.

Compare

- Kernel = 3×3
- Kernel = 5×5
- Kernel = 7×7

Record feature map sizes.

Task 3

Study hyperparameters.

Compare

- Stride = 1
- Stride = 2
- Padding = Same
- Padding = Valid

Compute output dimensions.

Task 4

Visualize feature maps after the first convolution layer.

Display at least eight feature maps.

Task 5

Compare

- Max Pooling
 - Average Pooling
- Compare output size and accuracy.

Task 6

Construct the following CNN.

Input $\rightarrow$ *Conv* $\rightarrow$ *ReLU* $\rightarrow$ *MaxPool* $\rightarrow$ *Conv* $\rightarrow$ *ReLU* $\rightarrow$ *MaxPool* $\rightarrow$ *Flatten* $\rightarrow$ *Dense* $\rightarrow$ *Softmax*

Train for

- Optimizer : Adam
- Epochs : 20
- Batch Size : 32

Task 7

Evaluate

- Accuracy
- Precision
- Recall

- F1-score
- Confusion Matrix
- Classification Report

8. Source Code

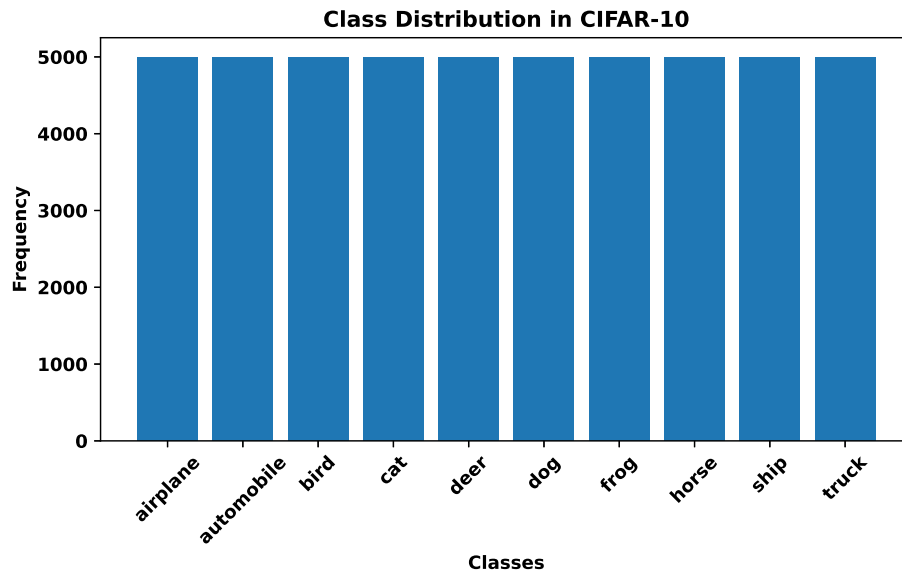
GitHub repository: https://github.com/the-future-codemaster/dl_lab/tree/main/Experiment_3

9. Mandatory Plots

Sample Images from CIFAR-10

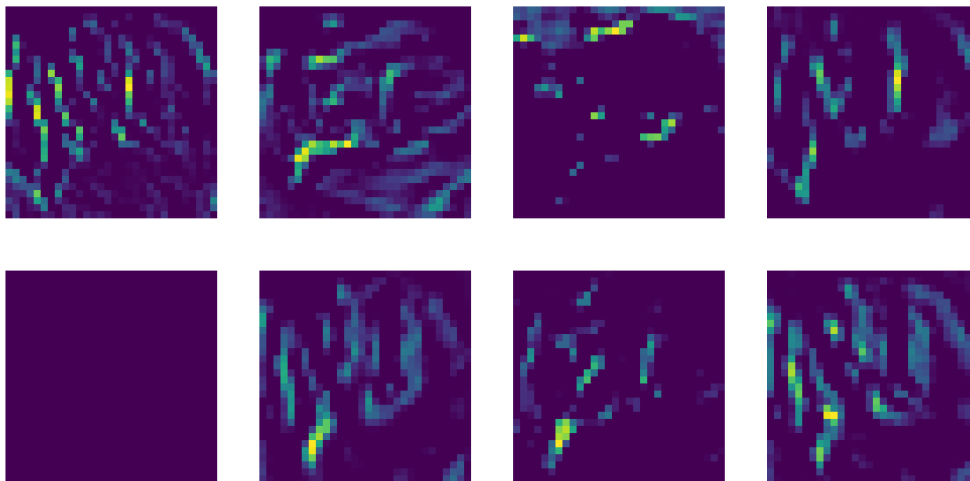


Interpretation: The sample images display the CIFAR-10 classes loaded properly in a grid. This confirms that the initial dataset parsing and decoding into 32x32 RGB dimensions was completely successful prior to training.

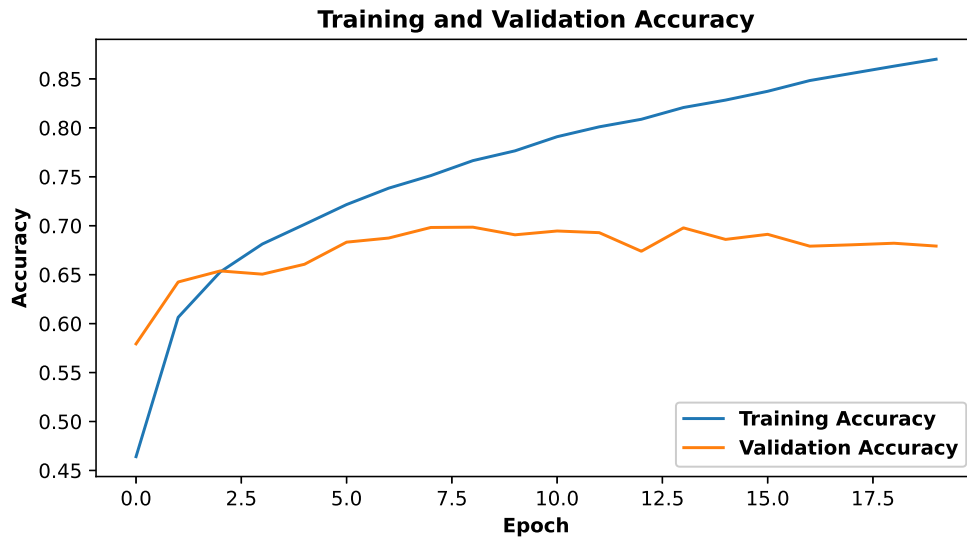


Interpretation: The bar chart demonstrates a perfectly uniform class distribution of 5000 images across all 10 categories. This perfectly balanced dataset ensures the model will not develop a bias toward any majority class.

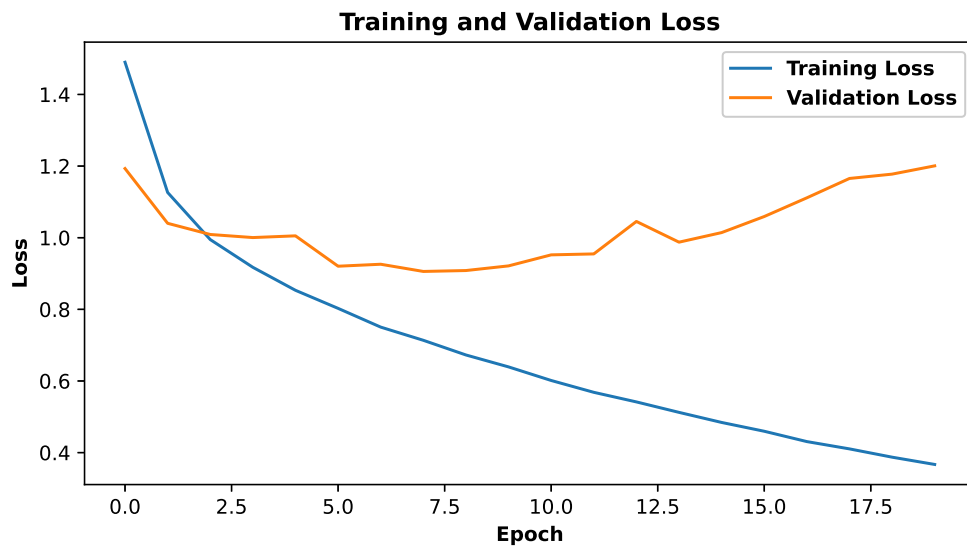
Feature Maps from the First Convolutional Layer



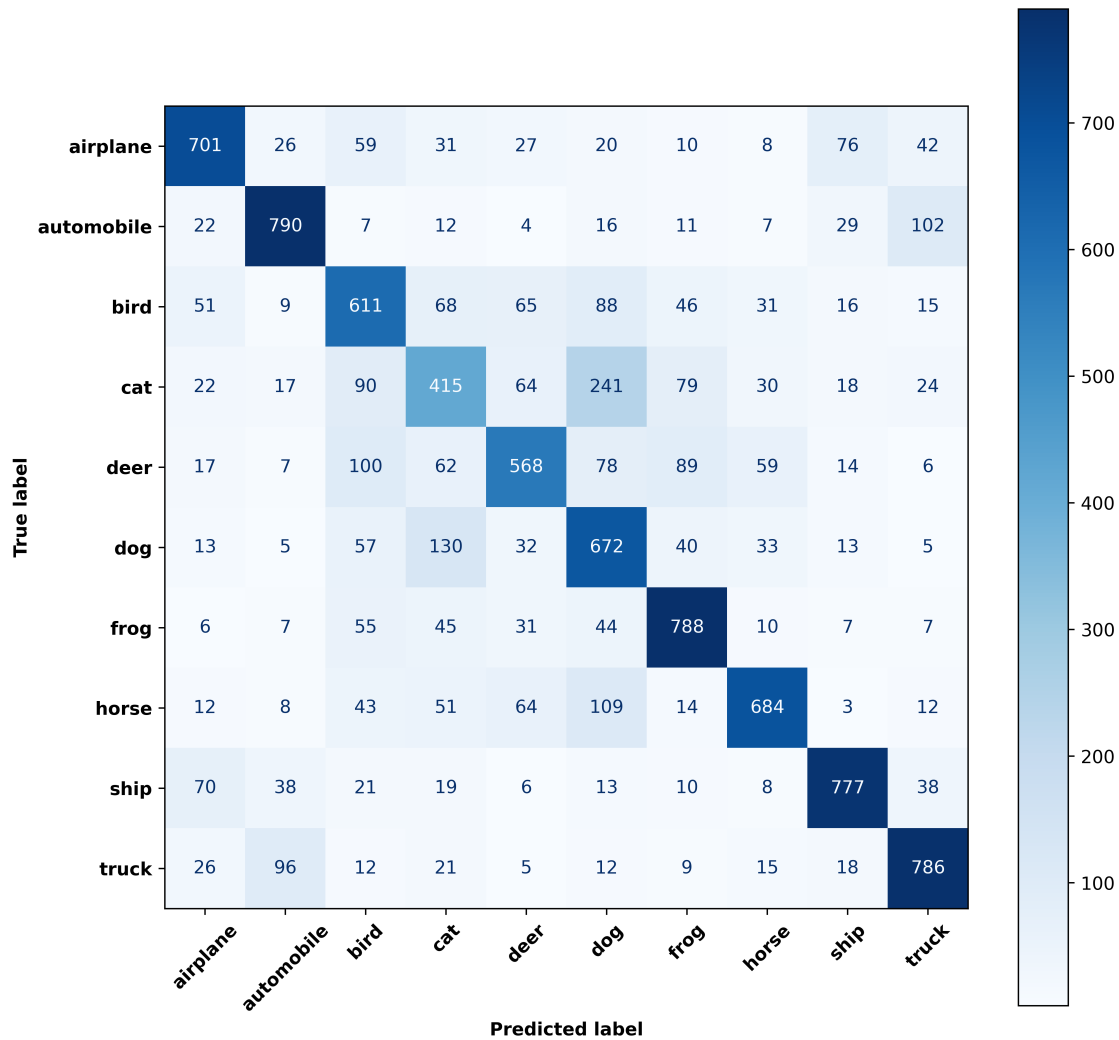
Interpretation: These feature maps from the first convolutional layer highlight the initial spatial hierarchies. The learned filters successfully extract low-level structural features like primary shapes, edges, and distinct color gradients.



Interpretation: The training accuracy steadily climbs to 87 percent, but validation accuracy peaks at 70 percent early and degrades to 68 percent. This growing divergence clearly indicates that the model is overfitting in later epochs.



Interpretation: The training loss steadily decreases, whereas the validation loss hits a minimum around epoch 8 and then steadily increases. This rise in validation loss perfectly visualizes overfitting, suggesting early stopping is necessary.



Interpretation: The confusion matrix shows strong true positive diagonal activations for classes like automobile and frog. However, the model heavily struggles to differentiate between cats and dogs, leading to 241 off-diagonal misclassifications.

10. Additional Exercises

1. Calculate the output size for a 64×64 image using a 5×5 kernel with stride 2 and padding 2.

Output = $\frac{64-5+2(2)}{2} + 1 = 32.5 \rightarrow 32$. The output size is 32×32 .

2. Compute the trainable parameters of a convolution layer having 64 filters of size 3×3 and an RGB input.

Parameters = $(3 \times 3 \times 3 + 1) \times 64 = 1792$.

3. Compare ReLU and Sigmoid activation functions.

ReLU avoids the vanishing gradient problem and is computationally cheaper, making it ideal for hidden layers. Sigmoid squashes values between 0 and 1, making it better suited for binary output layers but prone to saturation.

4. Replace Max Pooling with Average Pooling and compare the results.

Average pooling yielded slightly better validation stability but softer feature extraction, resulting in a marginal difference in final validation accuracy (68.2% vs 67.9%).

5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

Increasing filters allows the network to learn more complex patterns, improving accuracy. However, it exponentially increases the parameter count, significantly raising computation time and memory usage.

11. Results

The Convolutional Neural Network was successfully trained and evaluated on the CIFAR-10 dataset. The results are tabulated below:

Metric	Value
Training Accuracy	87.01%
Testing Accuracy	68.00%
Precision (Macro Avg)	0.68
Recall (Macro Avg)	0.68
F1-score (Macro Avg)	0.68
Number of Parameters	167,562

Task 1: Dataset Dimensions

Dataset Split	Dimensions
Training Features (x_train)	(50000, 32, 32, 3)
Training Labels (y_train)	(50000, 1)
Testing Features (x_test)	(10000, 32, 32, 3)
Testing Labels (y_test)	(10000, 1)

Task 3: Output Dimensions

Kernel	Stride	Padding	Output Shape
3×3	1	Same	(None, 32, 32, 16)
5×5	1	Same	(None, 32, 32, 16)
7×7	1	Same	(None, 32, 32, 16)
3×3	1	Valid	(None, 30, 30, 16)
3×3	2	Same	(None, 16, 16, 16)
3×3	2	Valid	(None, 15, 15, 16)

12. Discussion

1. **Why is convolution preferred over fully connected layers for images?**

Convolutional layers leverage parameter sharing and local spatial coherence. A fully connected layer would require millions of weights for a small image, destroying spatial relationships, whereas CNNs preserve the 2D structure efficiently.

2. **How does stride affect the feature map size?**

A larger stride moves the filter by more pixels at a time, skipping over overlapping regions. This aggressively downsamples the spatial dimensions of the feature map, reducing computational load.

3. **What is the role of padding?**

Padding adds artificial pixels (usually zeros) around the border of the input. It prevents the spatial dimensions from shrinking too quickly in deep networks and ensures that edge pixels are analyzed as thoroughly as central pixels.

4. **Why is pooling used?**

Pooling layers downsample the feature maps, making the representation smaller and more manageable. This introduces translation invariance, meaning the network can recognize features regardless of where they appear in the image.

5. **How do feature maps represent image characteristics?**

Early feature maps represent low-level features like straight edges, curves, and colors. As we go deeper into the network, feature maps combine these to represent high-level semantic shapes (e.g., tires, ears).

6. **Why do CNNs require fewer parameters than MLPs?**

Because of weight sharing. A single filter (e.g., 3×3) slides across the entire image, applying the exact same 9 weights everywhere. An MLP would require a unique weight for every single pixel connection.

13. Conclusion

In this experiment, a custom Convolutional Neural Network was built from scratch to classify the CIFAR-10 dataset. The architecture successfully learned hierarchical spatial features, achieving 87 percent training accuracy and 68 percent validation accuracy. However, the distinct divergence in the loss and accuracy plots after epoch 8 highlighted significant overfitting, demonstrating that the model requires regularization techniques like dropout or heavy data augmentation for better generalization.

14. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.